SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: ITA 0601 **COURSE NAME: MACHINE LEARNING**

COURSE OUTCOME COVERED

CO1: *Learning Problems, Perspectives and Issues, Concept Learning, Version Spaces & Candidate Elimination, Inductive Bias, Decision Tree Learning, Representation & Heuristic Search (BL1, BL2)*

Assessment Tool Weightage: 40%

QUESTIONS: 15 **Total Marks: 25**

Annexure A

APPLICATION BASED QUESTIONS

Code of Conduct:

I _S.Sri Sai Prasanna Durga (192424241) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: S.Sri Sai Prasanna Durga

SET-1:

1.A factory has three boxes containing colored balls. Box A contains 6 red, 4 blue, and 2 green balls; Box B contains 5 red, 7 blue, and 3 green balls; and Box C contains 8 red, 2 blue, and 5 green balls. One box is selected at random, but Box B is twice as likely to be chosen as each of the other boxes. A ball drawn from the chosen box is found to be blue. What is the probability that the selected box was Box C?

Code:

```
red_A = 6
blue_A = 4
green_A = 2
red_B = 5
blue_B = 7
green_B = 3
red_C = 8
blue_C = 2
green_C = 5
total_A = red_A + blue_A + green_A
total_B = red_B + blue_B + green_B
total_C = red_C + blue_C + green_C
P_A = 1 / 4
P_B = 2 / 4
P_C = 1 / 4
P_blue_A = blue_A / total_A
P_blue_B = blue_B / total_B
P_blue_C = blue_C / total_C
```

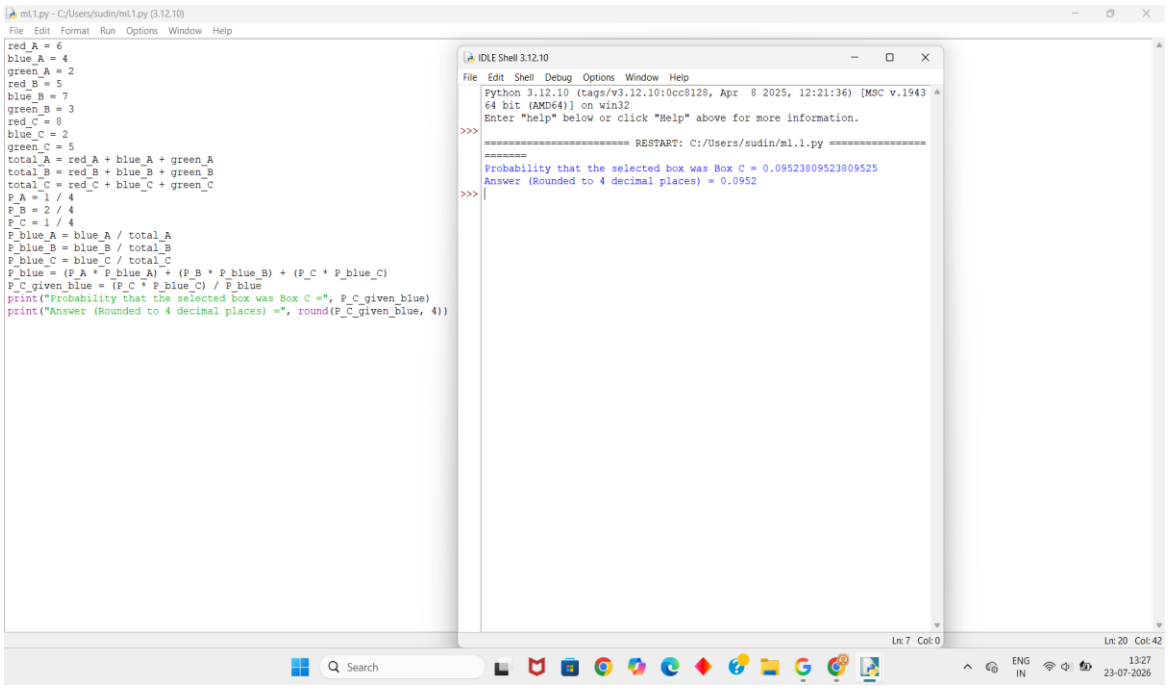
P_blue = (P_A * P_blue_A) + (P_B * P_blue_B) + (P_C * P_blue_C)

P_C_given_blue = (P_C * P_blue_C) / P_blue

print("Probability that the selected box was Box C =", P_C_given_blue)

print("Answer (Rounded to 4 decimal places) =", round(P_C_given_blue, 4))

Output:



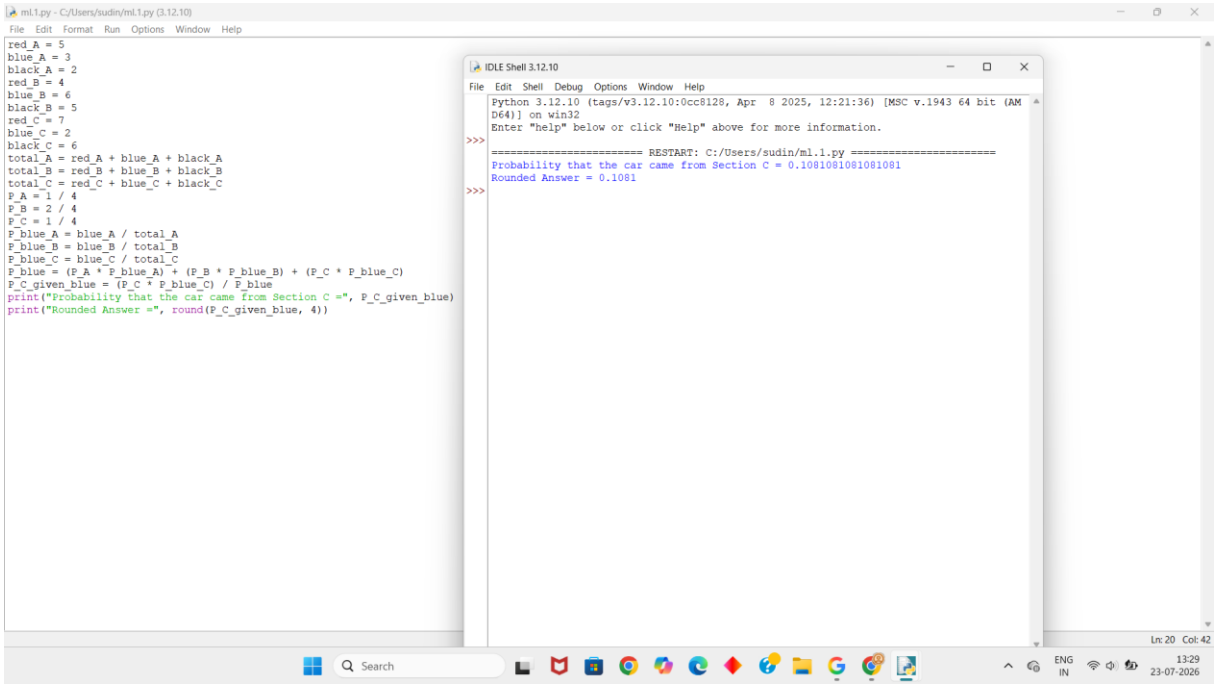
2. A parking facility has three sections containing cars of different colors. Section A has 5 red, 3 blue, and 2 black cars; Section B has 4 red, 6 blue, and 5 black cars; and Section C has 7 red, 2 blue, and 6 black cars. A section is chosen at random, but Section B is twice as likely to be selected as each of the other sections due to higher usage. A randomly selected car from the chosen section turns out to be blue. What is the probability that the car came from Section C?

Code:

red_A = 5

```
blue_A = 3
black_A = 2
red_B = 4
blue_B = 6
black_B = 5
red_C = 7
blue_C = 2
black_C = 6
total_A = red_A + blue_A + black_A
total_B = red_B + blue_B + black_B
total_C = red_C + blue_C + black_C
P_A = 1 / 4
P_B = 2 / 4
P_C = 1 / 4
P_blue_A = blue_A / total_A
P_blue_B = blue_B / total_B
P_blue_C = blue_C / total_C
P_blue = (P_A * P_blue_A) + (P_B * P_blue_B) + (P_C * P_blue_C)
P_C_given_blue = (P_C * P_blue_C) / P_blue
print("Probability that the car came from Section C =", P_C_given_blue)
print("Rounded Answer =", round(P_C_given_blue, 4))
```

Output:



3. In a real-world medical image classification task for detecting breast cancer, a deep-learning model integrated with the K-Nearest Neighbors algorithm is applied for binary classification (malignant vs. benign). The dataset consists of 1800 samples per class, with 20% of the data reserved for testing. During evaluation, the model reports 210 True Positives (TP), 190 True Negatives (TN), and 30 False Positives (FP). Based on the given information, determine the missing False Negatives (FN) value, and then calculate the Accuracy, Precision, Sensitivity (Recall), and Specificity of the classifier, explaining its effectiveness in medical diagnosis.

Code:

```
samples_per_class = 1800
test_per_class = samples_per_class * 20 / 100
TP = 210
```

TN = 190

FP = 30

actual_positive = test_per_class

FN = actual_positive - TP

accuracy = (TP + TN) / (TP + TN + FP + FN)

precision = TP / (TP + FP)

sensitivity = TP / (TP + FN)

specificity = TN / (TN + FP)

print("False Negatives (FN) =", FN)

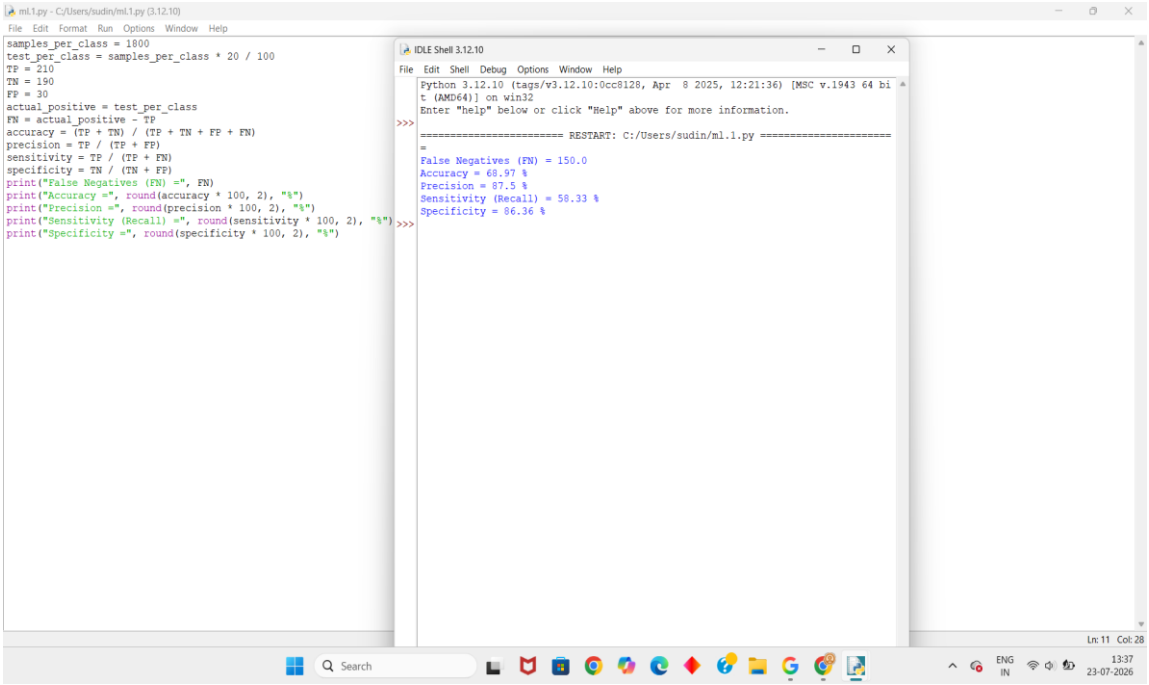
print("Accuracy =", round(accuracy * 100, 2), "%")

print("Precision =", round(precision * 100, 2), "%")

print("Sensitivity (Recall) =", round(sensitivity * 100, 2), "%")

print("Specificity =", round(specificity * 100, 2), "%")

Output:



4. Examine the performance of a machine-learning model using its training and validation loss values over multiple epochs:

a. Epoch 1: Training Loss = 2.8, Validation Loss = 2.4

a. Epoch 2: Training Loss = 2.5, Validation Loss = 2.2

(c) Epoch 3: Training Loss = 2.2, Validation Loss = 2.0

(d) Epoch 4: Training Loss = 2.0, Validation Loss = 2.1

(e) Epoch 5: Training Loss = 1.8, Validation Loss = 2.3

(f) Epoch 6: Training Loss = 1.6, Validation Loss = 2.6

(g) Epoch 7: Training Loss = 1.5, Validation Loss = 2.9

(h) Epoch 8: Training Loss = 1.3, Validation Loss = 3.2

i)Identify the epoch at which the model begins to overfit.

ii) Suggest two effective techniques to reduce overfitting in this model.

iii) Discuss how overfitting affects the model's generalization ability on unseen data.

Code:

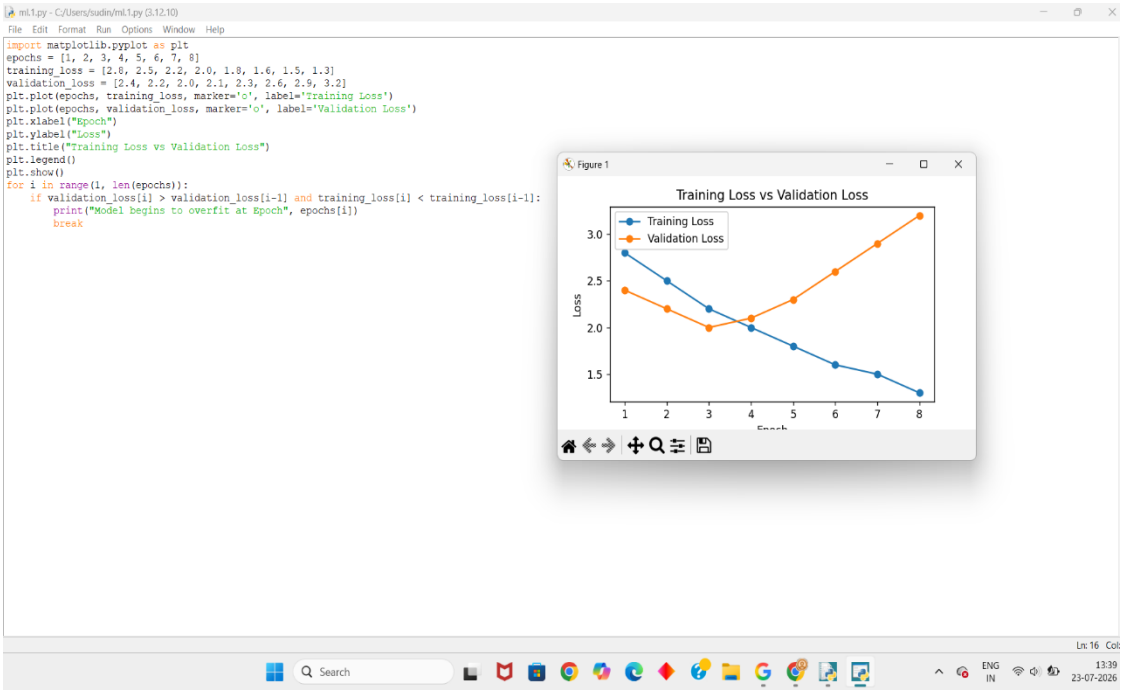
```
import matplotlib.pyplot as plt
epochs = [1, 2, 3, 4, 5, 6, 7, 8]
training_loss = [2.8, 2.5, 2.2, 2.0, 1.8, 1.6, 1.5, 1.3]
validation_loss = [2.4, 2.2, 2.0, 2.1, 2.3, 2.6, 2.9, 3.2]
plt.plot(epochs, training_loss, marker='o', label='Training Loss')
plt.plot(epochs, validation_loss, marker='o', label='Validation Loss')
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training Loss vs Validation Loss")
plt.legend()
plt.show()
for i in range(1, len(epochs)):
```

if validation_loss[i] > validation_loss[i-1] and training_loss[i] < training_loss[i-1]:

print("Model begins to overfit at Epoch", epochs[i])

break

Output:



5. Examine the performance of a machine-learning model using training and validation accuracy over multiple epochs:

(a) Epoch 1: Training Accuracy = 60%, Validation Accuracy = 58%

(b) Epoch 2: Training Accuracy = 65%, Validation Accuracy = 63%

(c) Epoch 3: Training Accuracy = 70%, Validation Accuracy = 68%

(d) Epoch 4: Training Accuracy = 75%, Validation Accuracy =

72%

(e) Epoch 5: Training Accuracy = 80%, Validation Accuracy = 73%

(f) Epoch 6: Training Accuracy = 85%, Validation Accuracy = 72%

(g) Epoch 7: Training Accuracy = 88%, Validation Accuracy = 70%

(h)Epoch 8: Training Accuracy = 92%,Validation Accuracy = 68%

i) Identify the epoch at which the model starts overfitting.

ii) Explain why validation accuracy decreases despite training accuracy increasing.

iii) Suggest two methods to improve validation performance.

Code:

```
import matplotlib.pyplot as plt
epochs = [1, 2, 3, 4, 5, 6, 7, 8]
training_accuracy = [60, 65, 70, 75, 80, 85, 88, 92]
validation_accuracy = [58, 63, 68, 72, 73, 72, 70, 68]
plt.plot(epochs, training_accuracy, marker='o', label='Training Accuracy')
plt.plot(epochs, validation_accuracy, marker='o', label='Validation Accuracy')
plt.xlabel("Epoch")
plt.ylabel("Accuracy (%)")
plt.title("Training Accuracy vs Validation Accuracy")
plt.legend()
plt.show()
for i in range(1, len(epochs)):
```

```
    if training_accuracy[i] > training_accuracy[i-1] and validation_accuracy[i] < validation_accuracy[i-1]:
        print("Model starts overfitting at Epoch", epochs[i])
        break
```

Output: